ParticleTrackingOF manual

Tomás Aquino

August 23, 2024

Abstract

This is a brief manual describing the code structure, main processes implemented, and how to compile and use the code on a UNIX system. ParticleTrackingOF is a C++ library for modeling advective–diffusive–reactive transport using Lagrangian particle tracking. It is based in part on OpenFOAM, a free and open source CFD software (https://www.openfoam.com/). It is designed to be used in conjunction with OpenFOAM for mesh generation and numerical determination of the underlying flow field. This offering is not approved or endorsed by OpenCFD Limited, producer and distributor of the OpenFOAM software and owner of the OPENFOAM® and OpenCFD® trade marks. For further details, see also the Doxygen documentation in `doc/Doxygen`.

1 Compilation and execution

The *ParticleTrackingOF* library is written in C++. Executables using the library must be compiled and linked to external dependencies locally using a version of the g++ or clang++ compiler compatible with C++17 and OpenMP (note that clang++ must be used on macOS due to linking issues with OpenFOAM). The code depends on the following external libraries:

- *OpenFOAM*: CFD software (https://www.openfoam.com/)
- *boost*: C++ library implementing useful features and extensions (boost.org)

These should be obtained and set up according to their respective documentations before using the present library. For compilation with clang++, llvm's OpenMP must be installed (`brew install libomp` on MacOS or `sudo apt install libomp-dev` on Linux).

The executables generated by this library are not OpenFOAM applications, although they rely heavily on the OpenFOAM API (`https://www.openfoam.com/documentation/guides/latest/api/`) and are designed to be used with previously-generated OpenFOAM meshes and underlying flow fields.

1.1 Compiling executables

The following compilable `.cpp` file is provided in the `src` directory:

- `ParticleTrackingOF.cpp` – Advective-diffusive-reactive particle tracking for stationary flow.

- `ParticleTrackingOF_TwoPhaseNonStationary.cpp` – Advective-diffusive-reactive particle tracking for time-dependent two-phase flow (with fixed mesh).

Parallel versions are also included:

- `ParticleTrackingOF_Parallel.cpp`
- `ParticleTrackingOF_Parallel.cpp`

By default, the `Makefile` provided for compiling the examples uses the C++ compiler obtained from the OpenFOAM environment variable `WM_COMPILER`. Note that the same compiler that was used for OpenFOAM must be used here; failure to do so will lead to dynamic linker errors at runtime. By default, the Makefile provided uses clang++ under a darwin (macOS) system and g++ under a linux system. OpenFOAM include directories are set by the variable `PTHOF`, which is set to `$FOAM_SRC` by default. The paths to provided header files are set in `PTH` (`../include` by default). These can be changed if necessary, and any additional include paths should be added to the `INC` variable. Executables must be linked to the OpenFOAM library. The corresponding path is set by the `PTHOFLIB` variable, which is set to `$FOAM_LIBBIN` by default and may also be changed if necessary. Alternatively, include and library directories can be added to the system `PATH` variable as usual. Finally, the base location for compiled executables is set in the `PTHBIN_BASE` variable (`../bin` by default). `PTHBIN` variable (`../bin /release` and `$PTHBIN_BASE/debug` by default for release and debug build; the default build type is release, and it can be changed by passing `BUILD=debug` to `make`).

Different models can be compiled by changing the `using namespace` declaration at the top of `main` in the `.cpp` file, which should be set to `using namespace ptof::model_name`, where `name` is the name of an existing model, defined in the corresponding namespace in `include/PTOF/Models.h`. Currently, the available models are:

- `advection_2d`
- `advection_diffusion_2d`
- `advection_diffusion_surface_decay_2d`
- `periodic_cartesian_advection_2d`
- `periodic_cartesian_advection_diffusion_2d`
- `periodic_cartesian_advection_diffusion_surface_decay_2d`
- `advection_3d`
- `advection_diffusion_3d`
- `advection_diffusion_surface_decay_3d`
- `periodic_cartesian_advection_3d`
- `periodic_cartesian_advection_diffusion_3d`
- `periodic_cartesian_advection_diffusion_surface_decay_3d`

- `bcc_cartesian_advection`
- `bcc_cartesian_advection_diffusion`
- `bcc_cartesian_advection_diffusion_surface_decay`
- `bcc_symmetryplanes_advection`
- `bcc_symmetryplanes_advection_diffusion`
- `bcc_symmetryplanes_advection_diffusion_surface_decay`
- `advection_diffusion_fpt`
- `periodic_cartesian_advection_diffusion_fpt`
- `bcc_cartesian_advection_diffusion_fpt`
- `bcc_symmetryplanes_advection_diffusion_fpt`

These names follow the following conventions:

- Models marked `2d` are two-dimensional, and models marked `3d` are three-dimensional.
- Models marked `surface_decay` include a linear fluid-solid surface decay reaction, with the solid phase not consumed ($A_F + A_S \rightarrow A_S$).
- Models marked `advection` include advective transport
- Models marked `diffusion` include diffusive transport
- Models marked `periodic_cartesian` support Cartesian boundary conditions, extracted directly from mesh data for boundaries that specify the periodic type.
- Models marked `bcc` are three-dimensional and exemplify the application of different types of periodic boundary conditions (cartesian or based on symmetry planes) for advective–diffusive and purely–advective transport in a body cubic centered beadpack.

Bash scripts to automate the model choice and compilation are provided. The script `set_model.sh`, to be run from a command line, should be followed by the model name. For example, running `./set_model.sh advection_diffusion` sets the namespace `ptof::model_advection_diffusion`. To compile the executables, open a command line, `cd` into the `src` directory, and run `./make.sh`, which takes an argument in the same way, plus an optional argument of `release` or `debug` to select the build type (by default, the build type is `release`). It both sets the model and compiles the corresponding executable. The resulting executable files are placed in the `bin/release` or `bin/debug`, where `build` is the build mode, and have the base name followed by an underscore and the model name (e.g., `ParticleTrackingOF_advection_diffusion_2d`). The script `make_all.sh` compiles all available models, and it takes an optional argument to set the build type. Parallel versions can be compiled directly with the makefile or by using the corresponding `_parallel.sh` scripts. Note that running `make ParticleTrackingOF` directly produces an executable named

ParticleTrackingOF using the currently set model in bin/release or bin/debug, and that running make.sh or make_all.sh will delete this file. Note also that if you would like to change the base executable folder, you can set the $PTHBIN_BASE variable in the make.sh and make_parallel.sh scripts; these scripts override the path set by the makefile.

If any of the shell scripts need to be given executing permission, this can be achieved as usual by running, e.g.,

```
chmod +x make.sh
```

1.2 Parameters and execution

The compiled executables take arguments themselves. Default values, if available, are used when '' is passed for the corresponding parameter. First, the stationary executable for simulations in series takes the following parameters (default in []):

- Cases directory [../cases]
- Name of case
- Name of transport parameter set
- Name of reaction parameter set
- Name of solver parameter set
- Name of initial condition parameter set
- Name of output parameter set
- Output directory [⟨Case directory⟩/output]
- Output file identifier [Based on parameter set names]
- Run number (nonnegative integer to index output) [None]

An executable named ExampleExecutable is run from a command line by executing ./ExampleExecutable followed by these parameters, separated by spaces. The case name refers to the directory with information about the problem to be solved, which should exist in the cases directory. Running the example executables followed by -h or –help prints a brief description and a list of the required parameters for the executable itself, and for each of the remaining five parameter sets (transport, reaction, solver, initial condition, and output). The required parameters for each of these five parameter sets should be placed in a file in the parameters directory within the case, named parameters_set_name.dat, where set is each of the sets (transport, reaction, etc.) and name is the corresponding name passed to the executable. In addition, the file of.dat, within the same directory, must contain:

- OpenFOAM cases directory [$FOAM_RUN]
- OpenFOAM case name [$FOAM_DIR]
- OpenFOAM case time [last OpenFOAM case time]

The OpenFOAM case and time are used to read the previously-generated mesh and flow field to be used in the transport simulation. Parameter values in these files should be placed one per line, except when specifically indicated. The character `#` indicates a comment: lines starting with this character are skipped, and parts of lines starting with this character are ignored.

Similarly, for the non-stationary executable in series, the parameters are

- Cases directory [../cases]
- Name of case
- Name of transport parameter set
- Name of phase parameter set
- Name of reaction parameter set
- Name of solver parameter set
- Name of initial condition parameter set
- Name of output parameter set
- Output directory [⟨Case directory⟩/output]
- Output file identifier [Based on parameter set names]
- Run number (nonnegative integer to index output) [None]

with an added parameter referring to the phase parameter set, which includes additional parameters about fluid phases. The corresponding parallel versions take an additional parameter:

- Number of parallel threads

2 Processes and code structure

The basic structure of the code relies on a generic CTRW object, which handles a set of Particle objects. Each Particle object is associated with two State objects, describing the particle state associated with the next and previous turning points. The dynamics of these particles are then handled by a Transitions object. Upon a transition, Transition objects take the two particle states, modify the newer state according to the current transition, and set to older state to the previous newer state. For the present use, the Transition object may include any combination of advection, diffusion, and reaction processes, as well as boundary conditions handled by a Boundary object. While the generic CTRW object interface is designed to handle more general dynamics, for the models considered here, each transition step is associated with a deterministic (although adaptive) time step.

2.1 Transport processes

Regarding transport, the ParticleTrackingOF library focuses on diffusion (with constant diffusion coefficient) and advection according to a heterogeneous flow field. The corresponding particle displacements are handled by a transport-related Transitions object relying on a TimeGenerator object for deterministic time steps. The time step is adaptive based on the local cell size, local advection, local diffusion, and local reaction rate. Within this object, particle displacements are handled by a JumpGenerator object, which produces a displacement given the current particle state. JumpGenerator objects implementing various methods for advective and diffusive steps. When both processes are present, a JumpGenerator is used to obtain the full displacement by summing the contributions from the respective JumpGenerators.

For diffusion, the following methods to compute displacement for a given time step are currently implemented:

- Stochastic forward-Euler temporal discretization of the Langevin diffusion equation.

All models provided use this option.

For advection, if the flow field is known analytically, methods to use the analytical flow field as a function of position are available in the ParticleTrackingOF library. However, the flow field must typically be obtained from separate flow simulations. The resulting flow field at an arbitrary set of points in the void space between beads is then to be provided by the user. The ParticleTrackingOF library allows for computing advective displacements using interpolation to obtain the flow field at arbitrary particle positions. The following interpolation procedures are currently implemented:

- Linear interpolation using a previously-generated OpenFOAM velocity field (based on OpenFOAM routines).

For a given method to compute the flow field at an arbitrary location, the following methods to compute advective displacement for a given time step are implemented:

- Forward-Euler temporal discretization
- Fourth-order Runge–Kutta temporal discretization

All models provided use the first option.

2.2 Boundary conditions

Boundary conditions are handled by a Boundary object, which takes the previous and current state upon a transport transition, and enforces the boundary conditions on the current state. Boundary conditions are associated with patches in a previously-generated OpenFOAM mesh. By default, the Boundary object handles the following boundary conditions:

- `reflecting`: Reflecting
- `reacting_reflecting`: Reacting and reflecting
- `periodic`: Periodic

- `absorbing`: Absorbing
- `custom`: Enforced by custom Boundary object
- `empty`: No effect (default for unspecified patches in mesh)

One of these types must be specified for any patches in the previously-generated OpenFOAM mesh with associated boundary conditions. This information is provided in a file `boundary_conditions_dynamics.dat` in the `boundary_conditions` directory of a case, where `dynamics` stands for different possible choices. Each line of the file should list a mesh patch name followed by one of the boundary condition types. Models not marked `fpt` obtain boundary conditions from the file `boundary_conditions_transport.dat`. Models marked `fpt` obtain them from the file `boundary_conditions_firstpassage.dat`. The first should set the appropriate boundary conditions for transport, whereas the second should change the first passage boundaries to `absorbing`. The following patch names hold special significance for this library:

- `inlet`: Flow inlet
- `outlet`: Flow outlet
- `wallFluidSolid`: Interface of carrier fluid (e.g., water) with solid phase (e.g., grains)
- `wallFluidFluid`: Interface of non-carrier fluid (e.g., air) with solid phase (e.g., grains)

The following types are implemented for periodic boundary conditions

- Periodic along cartesian dimensions
- Periodic according to prescribed symmetry planes

The `custom` condition has no effect for models not marked `fpt`. For models marked `fpt`, it reinjects particles according to the initial condition and is meant to be applied to the `outlet` patch (assuming it is not periodic). Additional customized boundary conditions can also be set up and implemented. All boundary conditions may also be used to store information in particle states when boundaries are hit or crossed. This is done through Store objects passed to the boundary, and Info objects within the State objects.

2.3 Reaction processes

Bulk reaction processes may be added by using a Transitions object which combines a transport-related Transitions object, as described above, with a reaction, and applies them sequentially. Surface (fluid–solid) reactions are handled by the Boundary object. The following surface reaction processes are currently implemented:

- Fluid-solid reaction on specified boundary patches: $A_{\text{solid}} + A_{fluid} \rightarrow A_{\text{solid}}$ and $A_{\text{solid}} + A_{fluid} \rightarrow \varnothing$.

The following types of initial distribution of solid reactants are currently implemented:

- `uniform`: Homogeneous over the solid phase.

2.4 Initial conditions

Initial conditions, or more precisely, the injection of particles into the domain, which may happen at any time, are handled by an InitialCondition object. The following initial conditions are implemented:

- `uniform`: Homogeneous throughout the domain
- `flux_weighted`: Flux-weighted throughout the domain
- `uniform_inlet`: Homogeneous at the inlet
- `flux_weighted_inlet`: Flux-weighted at the inlet
- `uniform_solid`: Homogeneous at the solid surface
- `uniform_near_solid`: Homogeneous at a fixed distance to the solid interface
- `uniform_cartesian_region`: Homogeneous in a cartesian box (possibly degenerate)
- `flux_weighted_cartesian_region`: Flux-weighted in a cartesian box (possibly degenerate)
- `prescribed_positions`: Prescribed initial particle positions
- `prescribed_positions_masses`: Prescribed initial particle positions and masses
- `prescribed_positions_masses_tags`: Prescribed initial positions, masses, and tags
- `uniform_inlet_continuous`: Continuous injection homogeneous at the inlet
- `flux_weighted_inlet_continuous`: Continuous injection flux-weighted at the inlet

2.5 Output

In the course of particle tracking simulations, quantities of interest relating to particle states can be computed, processed, and output. Typically, these quantities need to be measured a certain number times and/or when certain conditions are met, such as a given time being reached or a given control plane being crossed. The particle state quantities can be accessed for the current and old state of each particle through the CTRW object. By default, the Output class can make measurements with the following options:

- `step`: At fixed time intervals, starting at specified time
- `linear`: Fixed number of linearly-spaced measurements, starting and ending at specified times
- `log`: Fixed number of log-spaced measurements, starting and ending at specified times

It can also make measurements at the end of the simulation, and it controls the criterion for ending a simulation. Run the executable with the help flag to see a list of available end criteria and measurement types. Any number of measurement types may be chosen. Some measurement types involve particle tags, which are unique non-negative integer identifiers. Note that, if desired, fields must be previously computed and present in the OpenFOAM case.

Each of the previous output types is written to a separate file. By default, for the stationary simulations, the output file names have the structure:

```
Data_output_M_model_C_case_OF_ofcase_T_t_R_r_S_s_I_i_O_o_RUN_run.dat
```

where `output` is the output type, `model` is the model name, `case` is the simulation case name, `ofcase` is the OpenFOAM case name, `run` is the run number, and `t`, `r`, `s`, `i`, and `o` are the parameter set names for transport, reaction, solvers, initial condition, and output, respectively. If no run number is provided to the executable, this part of the filename is omitted. Similarly, for the non-stationary simulations, Each of the previous output types is written to a separate file. For the nonstationary simulations, the output file names have the structure:

```
Data_output_M_TwoPhaseNonstationary_model_C_case_OF_ofcase_T_t_
P_p_R_r_S_s_I_i_O_o_RUN_run.dat
```

where `p` is the parameter set name for phase information. As an alternative, if an output identifier is passed to the executable, the model to output portion of the file name is replaced by this identifier.